

ISS Live Tracker — Interview Q&A

Level: Beginner → Medium | **Questions:** 30 **Stack:** React.js, Node.js, Express.js, Three.js
(`react-globe.gl`), Open Notify API, OpenCage API, Nodemailer **Repo:**
https://github.com/ermadhav/ISS_Tracker

Important: These answers are based on your README and resume, not your source code. Change any answer so it matches what you actually built. If you didn't implement something (cooldowns, Haversine, database), say so honestly and explain how you would add it.

Check before the interview

- Open Notify's "ISS location now" endpoint returns only latitude, longitude and a timestamp. If your app shows **altitude and velocity**, be sure you know where they come from (another API, or computed from two positions). See Q15.
- Your README tells users to paste the OpenCage key into (`App.js`) and Gmail credentials into (`server.js`). Expect a question about this. The fix is environment variables (Q25).

Table of contents

1. [Basics and architecture](#)
 2. [React and the 3D globe](#)
 3. [APIs and data](#)
 4. [Location and email alerts](#)
 5. [Security, errors and deployment](#)
 6. [Improvements](#)
-

1. Basics and architecture

Q1. Explain this project in 30 seconds.

A: ISS Live Tracker shows the International Space Station's real-time position on a 3D globe, along with country, coordinates, altitude, velocity and the astronauts on board. It also embeds NASA's live video and a Stellarium sky view, and sends the user an email alert when the ISS passes near their location.

Q2. Why did you build it?

A: I wanted a project that combined live API data, a visual 3D interface and a real backend feature (email alerts), not just a static UI. Space data is also interesting enough to make the app fun to use and to demo.

Q3. What technologies did you use?

A:

- **Frontend:** React, `react-globe.gl` (Three.js underneath), Axios
- **Backend:** Node.js, Express, Nodemailer
- **APIs:** Open Notify (ISS location), OpenCage (reverse geocoding), Gmail SMTP (sending mail)

Q4. Describe the architecture and data flow.

A: The React frontend polls the ISS API every few seconds, updates the marker on the globe, and calls OpenCage to convert coordinates into a place name. When the user's location is close to the ISS, the frontend calls my Express backend, which sends the email through Nodemailer and Gmail SMTP.

Q5. What is an API, and what is a REST API?

A: An API lets one program talk to another. A REST API does this over HTTP using URLs and methods like GET and POST, and usually returns JSON. Open Notify is a simple REST API: I send a GET request and get JSON back.

Q6. Why did you need a Node/Express backend at all?

A: Sending email requires SMTP credentials, and these must never be in browser code because anyone could read them. The backend keeps the credentials private and does the sending. It's also the right place to add rate limits, caching, or calls to APIs that block browser requests.

2. React and the 3D globe

Q7. Why React for the frontend?

A: The UI updates constantly (position, country, coordinates), and React's state-driven rendering makes that simple: when state changes, the UI re-renders. It also has `react-globe.gl`, which wraps the 3D globe as a React component.

Q8. How do you fetch live data in React?

A: In a `useEffect`, I call the API with Axios and store the result with `useState`. To keep it live, I use `setInterval` to fetch again every few seconds.

```
jsx

useEffect(() => {
  const fetchISS = async () => { /* axios.get(...) then setPosition(...) */ };
  fetchISS();
  const id = setInterval(fetchISS, 5000);
  return () => clearInterval(id); // cleanup
}, []);
```

Q9. Why is the cleanup function (`clearInterval`) important?

A: Without it, the interval keeps running after the component unmounts, causing memory leaks and updates to a component that no longer exists. The cleanup function returned from `useEffect` stops the timer.

Q10. What is Three.js, and what is WebGL?

A: WebGL is the browser API for GPU-accelerated 3D graphics, but it's very low-level. Three.js is a JavaScript library that makes WebGL usable by giving me scenes, cameras, lights and meshes. `react-globe.gl` builds on Three.js and lets me render a globe with a few props.

Q11. How does the globe show the ISS position?

A: I pass the ISS's latitude and longitude as a data point (with optional altitude) to `react-globe.gl`, and the library places a marker on the sphere. When my state updates with a new position, the marker moves, and I can also rotate the camera to follow it.

Q12. How did you keep the app fast when updating every few seconds?

A: I only update the data that changes (the position), avoid recreating large objects on each render, and keep expensive work like geocoding out of the tightest loop. If it lagged, I'd use `useMemo`/`useCallback` and throttle updates.

3. APIs and data

Q13. Which APIs did you use, and for what?

A:

- **Open Notify:** the ISS's current latitude/longitude, and (if used) the list of people in space
- **OpenCage:** reverse geocoding (coordinates → country/state)
- **Gmail SMTP:** sending alert emails
- **NASA live video and Stellarium:** embedded as iframes

Q14. What does the Open Notify API return?

A: For the ISS location endpoint, it returns JSON with a `timestamp` and an `iss_position` containing `latitude` and `longitude`. There's a separate endpoint that returns the astronauts currently in space.

Q15. Where do altitude and velocity come from?

A: (*Answer with what your code does.*) Open Notify doesn't provide them. Options: another API (for example `wheretheiss.at` returns altitude and velocity), or compute velocity from two consecutive positions and their timestamps (distance divided by time). Tell them which one you used. If you used a fixed value, say so.

Q16. What is reverse geocoding?

A: Geocoding turns an address into coordinates, and reverse geocoding does the opposite: it turns latitude/longitude into a place name. I use OpenCage for that so the UI can show the country and state the ISS is passing over.

Q17. What happens when the ISS is over the ocean?

A: The geocoding API returns no country or an empty result. I handle that by showing a fallback like "Over the ocean" instead of an error or blank text.

Q18. Why use Axios instead of `fetch`?

A: Both work. Axios gives automatic JSON parsing, simpler error handling for HTTP errors, interceptors and timeouts. `fetch` is built in and needs no library. It's a convenience choice.

Q19. What is CORS, and did you run into it?

A: CORS (Cross-Origin Resource Sharing) is a browser rule that blocks a page from reading responses from another origin unless that server allows it. My frontend and backend run on different ports/origins, so the Express backend needs the `cors` middleware. Also, if an API only supports plain HTTP, a page served over HTTPS can't call it (mixed content), which is one reason to route such calls through the backend.

Q20. How do you deal with API rate limits?

A: Free APIs like OpenCage limit requests per day. I avoid calling the geocoder on every tick: only when position changed enough, or every N seconds, and I cache the last result. The ISS moves around 7.66 km/s, so a small delay in place names is fine.

4. Location and email alerts

Q21. How do you get the user's location?

A: With the browser's Geolocation API, `navigator.geolocation.getCurrentPosition()`, after the user grants permission. Or the user can enter a city, which I geocode. I handle denial with a fallback message.

Q22. How do you decide the ISS is "near" the user?

A: I compute the distance between the ISS coordinates and the user's coordinates and trigger the alert when it's below a threshold. On a globe, the correct measure is the **Haversine formula**, because plain Euclidean distance on lat/long is wrong, especially near the poles.

js

```
function haversineKm(lat1, lon1, lat2, lon2) {
  const R = 6371; // Earth radius in km
  const toRad = (d) => (d * Math.PI) / 180;
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(a));
}
```

Q23. How do the email alerts work?

A: The frontend sends a request to my Express endpoint with the user's email. The backend uses Nodemailer with a Gmail SMTP transporter to send the message.

Q24. What is SMTP, and why did you need a Gmail App Password?

A: SMTP (Simple Mail Transfer Protocol) is the standard protocol for sending email. Gmail doesn't allow normal account passwords for SMTP from apps. With 2-step verification enabled you can create an **App Password**, a 16-character password just for that app that can be revoked independently.

5. Security, errors and deployment

Q25. Your README says to put API keys directly in the code. Is that a problem?

A: Yes. Keys and passwords in source files end up in Git history, and anyone with repo access can use them. The correct approach is environment variables: use a `.env` file with `dotenv` on the backend, add `.env` to `.gitignore`, and rotate any key that was ever committed. In React, remember that any key in frontend code is visible to users, so sensitive keys belong on the backend.

Q26. How do you handle API failures?

A: `try/catch` around every request, keep showing the last known position, and show a small error or "reconnecting" state. One failed call shouldn't crash the UI, and I don't want to spam retries either.

Q27. How do you stop the same user getting emails repeatedly?

A: The ISS stays in range for several minutes, so without protection the user would get many emails. I'd store a "last alert sent" timestamp per user and enforce a cooldown, for example one email per pass. *(If you haven't built this, say it's a known improvement.)*

Q28. How did you embed NASA's live video and Stellarium?

A: With `<iframe>` elements pointing to the embed URLs. I set a sensible size, `title` and `allow` attributes, and I'm aware that iframes load third-party content, so I only embed

trusted sources.

Q29. How would you deploy it?

A: The React frontend on Vercel or Netlify, and the Express backend on Render or Railway. Secrets go in the hosting platform's environment variable settings, and I set the backend's CORS to allow only my frontend's domain.

6. Improvements

Q30. What would you improve or add next?

A:

- Move all secrets to environment variables.
- Store user subscriptions in a database (MongoDB or PostgreSQL) so alerts work even when the tab is closed.
- Use a scheduled job (cron) plus **orbit prediction** (for example `satellite.js` with TLE data) to tell users *in advance* when a visible pass will happen, instead of only reacting to the current position.
- Add a cooldown and unsubscribe link for emails.
- Add tests for the distance function and the API handling code.